

# **Informe Analítico Usando MILP de la Propuesta de Optimización del Algoritmo SHA-3 Keccak para Hardware Limitado**

**Escuela de Ciencias de la Computación**

***Seguridad de Sistemas Informáticos - CC411A***



***Mg. Manuel Alejandro Quispe Torres***

**Universidad Nacional de Ingeniería**

**Leonardo Alexander Chacón Roque - 20221002K**

**11 de julio del 2026**

# 1. INTRODUCCIÓN

Keccak es la función de esponja en la que se basa SHA-3, estandarizada en FIPS 202 (NIST, 2015). Su núcleo, Keccak-f[1600], ejecuta 24 rondas fijas de cinco pasos,  $\theta$ ,  $\rho$ ,  $\pi$ ,  $\chi$ ,  $\iota$ , sobre un estado de  $5 \times 5$  palabras de 64 bits. El único paso no lineal es  $\chi$ , una S-box de fila de 5 bits aplicada 320 veces por ronda; la resistencia diferencial del diseño depende de cuántas S-boxes se ven forzadas a estar activas en el mejor trail que un atacante pueda construir.

Este informe cubre los objetivos, las dos variables dinámicas y su rol en la esponja, el rediseño estructural y su costo de hardware, el modelo MILP, los experimentos y resultados, el canal lateral, y la conclusión. Cada afirmación de seguridad o de costo se acompaña de una medición verificable en `implementacion.ipynb` o `MILP_AES_Keccak_modificado.ipynb`, no de una descripción cualitativa. El peso mínimo de un trail acota directamente la probabilidad diferencial explotable,  $2^{-\text{peso}}$ , y el número de pares de texto plano necesarios para un ataque, aproximadamente  $2^{\text{peso}}$ : esta es la métrica central del informe.

## 2. OBJETIVOS

1. Incorporar dos variables dinámicas a la permutación, nivel de seguridad y dominio de la aplicación, incluyendo la posibilidad de modificar capacidad o ratio de la esponja, sin introducir debilidades de canal lateral.
2. Rediseñar la capa lineal, sustituyendo `theta+rho+pi` por difusión basada estrictamente en XOR y rotaciones, alcanzando al menos 80% de difusión de lanas en significativamente menos rondas que el estándar.
3. Revisar el paso no lineal `chi`, encontrando y refutando matemáticamente 2 vulnerabilidades lógicas propuestas.
4. Demostrar formalmente, mediante un modelo MILP que compara línea base y modificación, que el diseño resultante mantiene un número óptimo de S-boxes activas, linealizando tanto `chi` con la tabla de distribución de diferencias como con variables AND.

## 3. DESCRIPCIÓN DEL ALGORITMO MODIFICADO

### 3.1. Primera variable dinámica: `intentos_fallidos`

`intentos_fallidos` es un contador global mutable, incrementado por `registrar_intento_fallido()`, del que dependen cuatro aspectos de `keccak_f1600_variante`:

```

def keccak_f1600_variante(state):
    rondas = min(24, 8 + intentos_fallidos)
    for round_idx in range(rondas):
        if intentos_fallidos < 3:
            state = theta(state); state = rho(state)
            state = pi(state); state = chi(state)
        else:
            state = theta(state); state = rho(state)
            state = chi(state); state = pi(state)
            state = sigma(state, intentos_fallidos)
        state = iota(state, round_idx, intentos_fallidos, dominio_aplicacion)
    return state

```

Es decir: las rondas equivalen a  $\min(24, 8 + \text{intentos\_fallidos})$  en vez de 24 fijas; luego, el orden: con  $\text{intentos\_fallidos} < 3$  se usa  $\theta \rightarrow \rho \rightarrow \pi \rightarrow \chi \rightarrow \iota$ , el orden clásico; si no, se usa  $\theta \rightarrow \rho \rightarrow \chi \rightarrow \pi \rightarrow \sigma \rightarrow \iota$ , con  $\chi$  y  $\pi$  intercambiados y un paso nuevo  $\sigma$ ; además,  $\sigma$  es un paso nuevo que hace  $\text{state}[x][y] \wedge= \text{intentos\_fallidos} \ll (x+y)$  y rota el resultado  $(\text{intentos\_fallidos} + x + y) \% 64$  bits; también, la constante de ronda de  $\iota$  pasa de  $\text{RC}[i]$  a  $\text{RC}[i] \oplus \text{intentos\_fallidos} \oplus \text{dominio\_aplicacion}$ , el segundo término se explica a continuación. `sha3_256_variante` reutiliza la misma esponja, con padding  $\text{pad}_{10 \times 1}$  y absorción de  $\text{rate\_bytes}$  por bloque y exprimido de 32 bytes, que `sha3_256_fips`, solo cambiando la permutación interna. Forzar 5  $\text{intentos\_fallidos}$  sobre el mismo mensaje produce un hash distinto al original, evidenciando no conmutatividad del reordenamiento; esto no dice nada sobre resistencia diferencial, que es lo que mide el modelo MILP.

### 3.2. Segunda variable dinámica: dominio\_aplicacion

Se incorpora una segunda variable dinámica, de dominio de la aplicación, implementada como  $\text{dominio\_aplicacion} \in \{0, 1, 2, 3\}$ , con 0=general, 1=IoT/sensor, 2=tarjeta inteligente, 3=firmware embebido crítico, combinada por XOR en `iota` junto a  $\text{intentos\_fallidos}$ , sin afectar rondas ni orden de pasos. Por ser una constante conocida para una ronda dada, que no depende del mensaje, no altera la propagación diferencial: para un par  $a, b$  y una constante  $c$  cualquiera,  $(a \oplus c) \oplus (b \oplus c) = a \oplus b$ , argumento que se formaliza más adelante junto al modelo MILP. Se verificó que, con  $\text{intentos\_fallidos}=5$  fijo, variar  $\text{dominio\_aplicacion}$  produce 4 hashes distintos, un efecto real, sin cambiar el número de rondas ni el orden.

### 3.3. Capacidad y ratio dependientes del dominio

`dominio_aplicacion` cumple también el rol de modificar capacidad o ratio de la esponja como mecanismo de inyección dinámica, seleccionando uno de los 4 pares oficiales, ratio y capacidad, de la familia SHA-3, donde ratio más capacidad suman siempre 1600 bits, en vez de valores inventados:

**Tabla 1.***Perfiles de ratio/capacidad por dominio\_aplicacion*

| dominio_aplicacion                | Ratio     | Capacidad | Cota de seguridad mín(cap/2, 256) |
|-----------------------------------|-----------|-----------|-----------------------------------|
| 0, general, SHA3-256, por defecto | 1088 bits | 512 bits  | 256 bits                          |
| 1, IoT/sensor, SHA3-224           | 1152 bits | 448 bits  | 224 bits                          |
| 2, tarjeta inteligente, SHA3-384  | 832 bits  | 768 bits  | 256 bits                          |
| 3, firmware crítico, SHA3-512     | 576 bits  | 1024 bits | 256 bits                          |

Cota genérica de esponja:  $\min(\text{capacidad}/2, \text{salida})$  (Bertoni et al., 2007). El dominio 0 preserva el comportamiento ya validado antes, con el mismo hash de referencia; los otros 3 producen hashes distintos.

El hallazgo cuantificado, no ocultado, es que el perfil 1, IoT/sensor, cede margen de seguridad genérico a 224 bits, el mismo balance de SHA3-224 estándar, mientras 2 y 3 mantienen 256 bits porque su capacidad ya excede el doble de la salida. Este análisis es sobre la esponja, la capa externa, e independiente del modelo MILP, que analiza la permutación interna y no depende de ratio ni capacidad.

## 4. REDISEÑO ESTRUCTURAL: CAPA LINEAL Y PASO NO LINEAL

### 4.1. Nueva capa de difusión y su costo

Reemplazar `theta+rho+pi` por difusión basada estrictamente en XOR y rotaciones, con al menos 80% de difusión de lanas en significativamente menos rondas que el estándar, es el eje del rediseño estructural. `capa_lineal_rediseño` hace, para las filas, eje Y, lo mismo que `theta` hace para las columnas, eje X, y difunde cada resultado con una rotación distinta por celda (x,y), reutilizando la tabla `ROTATION_OFFSETS`:

$$\begin{aligned}
 C[x] &= \oplus_y \text{state}[x][y] & E[y] &= \oplus_x \text{state}[x][y] \\
 \text{state}'[x][y] &= \text{state}[x][y] \oplus \text{ROTL}(C[x-1] \oplus \text{ROTL}(C[x+1], 1), \text{OFFSETS}[x][y]) \oplus \\
 &\quad \text{ROTL}(E[y-1] \oplus \text{ROTL}(E[y+1], 2), \text{OFFSETS}[x][y])
 \end{aligned}$$

Un primer intento, con la misma rotación para las 5 lanas de destino, igual que `theta` original, se descartó: solo activa un número absoluto de bits casi constante sin importar  $z$ , unos 11 bits tras 1 ronda tanto a  $z=4$  como a  $z=64$ , por lo que su difusión porcentual se diluye al crecer  $z$ , 5% a 64 bits contra 81% a 4 bits. Variar la rotación por celda expande un bit a todas las lanas y escala correctamente. Se verificó que `capa_lineal_rediseño` es una permutación válida, con rango GF(2) completo por Gauss-Jordan: 1600/1600 a 64 bits, 100/100 a  $z=4$  y 200/200 a  $z=8$ .

La difusión se mide como fracción de las 25 lanas tocadas por un bit activado, la noción de Keccak, no la fracción de bits que cambian de valor, que para cualquier lineal bien

mezclada converge aproximadamente a 50% y nunca a 80-100%, por lo que no es la métrica relevante aquí:

**Tabla 2.**

*Difusión de lanas en 1 ronda: rediseño vs. estándar theta+rho+pi*

| <b>z</b> | <b>Rediseño</b> | <b>Estándar</b> |
|----------|-----------------|-----------------|
| 4        | 92%             | 44%             |
| 8        | 99.8%           | 44%             |
| 64, real | 100%            | 44%             |

El estándar alcanza el mismo nivel recién en la ronda 2 o 3, ver Tabla 4. `capa_lineal_rediseño` cumple el umbral de 80% en 1 sola ronda en los tres anchos probados.

Para no quedarse en un argumento cualitativo, se instrumentaron las operaciones XOR, AND y NOT para contar compuertas, tratando AND-NOT como una sola celda fusionada, estándar en síntesis ASIC o FPGA, y se validó el conteo con síntesis real de hardware, Yosys 0.33 más ABC, mapeando a compuertas AND/XOR/NAND/NOR/OR/AND-NOT y midiendo profundidad como camino topológico más largo. `theta` y `chi` coinciden exactamente con el conteo manual; `capa_lineal_rediseño` resulta 8.2% más cara, 6926 en vez de 6400 compuertas, con profundidad real 12, no 10, porque la etapa de filas depende de la salida completa de la etapa de columnas, no en paralelo:

**Tabla 3.**

*Costo de hardware para z=64, verificado por síntesis, y costo total a margen de seguridad equivalente.*

| <b>Componente</b>                                       | <b>Compuertas</b> | <b>Profundidad</b> |
|---------------------------------------------------------|-------------------|--------------------|
| theta, rho y pi son libres: rotación y permutación fija | 3200              | 6                  |
| capa lineal rediseño                                    | 6926, 2.16×       | 12, 2.00×          |
| chi, AND-NOT fusionado                                  | 3200              | 2                  |
| Ronda completa, estándar y rediseño                     | 6400 / 10126      | 8 / 14             |

El costo por ronda de rediseño es 1.582 veces el de estándar. Comparado con la razón de pesos de la Tabla 4, hay un cruce: rediseño rinde más margen de seguridad por compuerta a 2-3 rondas, 3.6× a 11×, y menos desde 8 rondas, 1.07× a 1.40×. Ese ahorro, junto con el de latencia, no se cobra aún en esta implementación.

## 4.2. Paso no lineal chi: dos intentos refutados

Se intentaron dos simplificaciones de `chi` para abaratar hardware, ambas refutadas con prueba matemática. La primera, `chi_sin_not`, quita el NOT:  $state[x] \oplus (state[x+1] \wedge state[x+2])$ . Por fuerza bruta sobre los 32 patrones de 5 bits, no es biyectiva, rompería la permutación de Keccak-f. Descartada.

La segunda, `chi_parcial`, aplica `chi` solo a la mitad de los bits del carril, con identidad en el resto. Invirtiendo `capa_lineal_rediseño` se construyó una diferencia con un bit activo en una posición saltada, y se verificó exhaustivamente, todos los  $2^z$  valores a  $z=4,8$ , y 20 000 muestras a 64 bits, que se propaga de forma determinista, probabilidad 1 y peso 0, una característica diferencial de 1 ronda con probabilidad 1, una vulnerabilidad crítica. Descartada.

Se mantiene `chi` sin modificar: ambos intentos refutados son las 2 vulnerabilidades lógicas buscadas y la ecuación correcta en ambos casos es la `chi` estándar ya usada.

## 5. MODELO MILP

Implementado íntegramente en Python con PuLP (Mitchell et al., 2011), en `MILP_AES_Keccak_modificado.ipynb`, en vez de SageMath o Colab, ya que no depende de licencia ni de Colab, trae CBC empaquetado y usa Gurobi disponible.

### 5.1. Reducción de escala

El ancho real, 64 bits, haría el MILP intratable; se usa  $z$  como parámetro sobre las mismas tablas RC y ROTATION\_OFFSETS, tomadas módulo  $z$ .  $z=4$  es Keccak-f[100] y  $z=8$  es Keccak-f[200], instancias oficiales de la familia, con 20 o 40 S-boxes  $\chi$  por ronda.

### 5.2. Constantes XOR transparentes a $\Delta$

Un trail sigue  $\Delta = a \oplus b$  para un par relacionado. Si un paso hace XOR a ambos lados con la misma constante  $c$ , que no depende de los datos,  $(a \oplus c) \oplus (b \oplus c) = a \oplus b$ :  $\Delta$  no cambia. Esto aplica a `iota`, cuya constante es `RC[i] * intentos_fallidos * dominio_aplicacion`, y al término XOR de `sigma`; solo la rotación de `sigma` sí actúa sobre  $\Delta$ . En consecuencia, el MILP no necesita variables para esos términos: solo `theta` o `capa_lineal_rediseño`, `rho`, la rotación de `sigma` y `chi` entran al modelo.

### 5.3. Variables, linealización de $\chi$ y función objetivo

Cada bit de diferencia es una variable binaria de PuLP. `rho`, `pi` y las rotaciones, incluidas las de `capa_lineal_rediseño`, son permutaciones puras, sin variables nuevas. `theta` y

las reducciones de `capa_lineal_rediseño` combinan bits por XOR y usan la linealización estándar de 3 variables:  $c \leq a+b$ ,  $a \leq b+c$ ,  $b \leq a+c$ ,  $a+b+c \leq 2$ .

$\chi$ , definida como  $\text{chi\_row}(a)[x] = a[x] \wedge ((1-a[(x+1)\%5]) \& a[(x+2)\%5])$ , se linealiza con un selector one-hot exacto sobre la tabla de distribución de diferencias, DDT, calculada por fuerza bruta,  $2^5 \times 2^5 = 1024$  casos: 316 transiciones válidas con entrada no nula, pesos observados de 2, 3 y 4, mínimo 2, ya que no existe transición de peso 1. Es un modelo exacto, no una relajación tipo branch number.  $\chi$  se linealiza además con variables AND: se implementó también, ya que NOT es afín y el término se reduce a un AND de las diferencias rotadas, linealizado con  $w = \text{OR}(da, db)$  y  $t \leq w$ ; reproduce el óptimo, peso 2, en 3 de 4 casos de validación, pero sobreestima en general, por ejemplo 5 contra el peso real 3 para el patrón 01011, porque linealiza cada término independientemente sin capturar condiciones compartidas. Se usa la DDT para los resultados finales por ser exacta y estar verificada por un segundo método independiente, la inversión GF(2); la versión AND queda implementada como evidencia del cumplimiento del requisito literal.

El objetivo minimiza la suma de pesos, logaritmo negativo base 2 de la probabilidad, de las transiciones activas, no un conteo puro de filas, ya que es la cantidad que determina la probabilidad diferencial real. El solver se prioriza Gurobi, luego HiGHS, luego CBC; en este entorno solo CBC y HiGHS estaban disponibles, y los resultados principales se obtuvieron con HiGHS (Huangfu y Hall, 2018). Se detectó que PuLP puede reportar estado óptimo aunque el solver solo cortó por tiempo, visto con CBC; se mitigó parseando el log nativo del solver, y los resultados con HiGHS quedan además respaldados por la verificación algebraica independiente.

## 6. EXPERIMENTOS

Se ejecutó `build_trail_model(z, num_rounds, intentos_fallidos, time_limit=120)` para tres configuraciones: una grilla de validación de 1 ronda con MILP exacto,  $z$  en 4 u 8, para los dos órdenes de pasos, con `intentos_fallidos=0` para el clásico e `intentos_fallidos=5` para el variante; luego, una grilla de 2 y 3 rondas por búsqueda constructiva para las mismas combinaciones, más `capa_lineal_rediseño` con  $\chi$  estándar; finalmente, una grilla ligada a los rangos de `intentos_fallidos` solicitados, menor a 10, menor a 20 y menor a 30, usando `rondas = min(24, 8+intentos_fallidos)`. Los rangos menor a 20 y menor a 30 saturan ambos en 24 rondas, ya que `intentos_fallidos ≥ 16`, por lo que se usó un único valor representativo, 19, para ambos, y dos valores, 0 y 9, para exhibir los extremos del rango menor a 10, el único no saturado.

## 7. RESULTADOS

### 7.1. Una ronda

El MILP exacto converge y prueba optimalidad para 1 ronda en las 4 combinaciones de  $z$  y orden, con peso mínimo de 2 en todos los casos, entre 5.5 y 44.3 segundos de solver con HiGHS. Esto se verificó además algebraicamente, sin depender del solver:  $\theta$  y  $\rho$  son biyecciones GF(2)-lineales invertibles; se construyó explícitamente la inversa de  $\theta \circ \rho$ , por Gauss-Jordan sobre GF(2), para hallar, dado un patrón de un único bit activo justo antes de  $\chi$ , el único estado previo que lo produce. Como el peso mínimo de una fila  $\chi$  activa es 2, el peso mínimo de un trail de 1 ronda es exactamente 2 para cualquier  $z$  y orden; el reordenamiento  $\chi \leftrightarrow \pi$  no lo cambia, porque  $\pi$  solo reubica lanes sin alterar bits activos por fila. La misma técnica, aplicada a `capa_lineal_rediseño`, confirma que el mínimo de 1 ronda tampoco cambia con el rediseño estructural, ya que depende solo de la DDT de  $\chi$ , no de qué capa lineal invertible se use. Peso 2 implica probabilidad  $2^{-2}=1/4$ , aproximadamente 4 pares de texto plano para un ataque de 1 ronda.

### 7.2. Dos a más rondas

Más allá de 1 ronda, el MILP exacto no converge en tiempo práctico, con la cota inferior estancada en 0 tanto con CBC como con HiGHS, una limitación conocida de codificaciones one-hot en MILP genérico, no un error del modelo. Se construyó en su lugar un trail real y verificable: invertir la capa lineal,  $\theta \circ \rho$  o `capa_lineal_rediseño` según el caso, para fijar el estado inicial, y propagar eligiendo en cada fila activa la transición de menor peso de la DDT.

Se intentó además fortalecer el MILP con cortes no-good explícitos, una restricción lineal por cada una de las 676 combinaciones inválidas entrada-salida de la DDT de  $\chi$ , sumada al selector one-hot. Esto confirma el óptimo de 1 ronda, peso 2, y para  $z=4$ , 2 rondas, el solver reportó estado Optimal con peso 8, justo en el límite de 1500 s configurado; se trata como cota, no como prueba certificada, por el mismo motivo ya señalado sobre reportes prematuros de optimalidad, pero coincide exactamente con la cota ya construida por búsqueda greedy, indicio de que esa cota podría ser el óptimo real.

**Tabla 4.**

*Cotas superiores de peso de trail: estándar, variante dinámica y rediseño estructural*

| $z$ | rondas | estándar, clásico | variante dinámica | rediseño estructural |
|-----|--------|-------------------|-------------------|----------------------|
| 4   | 2      | 8                 | 20                | 45                   |
| 4   | 3      | 29                | 70                | 103                  |
| 8   | 2      | 8                 | 21                | 88                   |
| 8   | 3      | 31                | 93                | 213                  |
| 4   | 8      | 315               | 367               | 393                  |

| z | rondas | estándar, clásico | variante dinámica | rediseño estructural |
|---|--------|-------------------|-------------------|----------------------|
| 4 | 17     | 840               | 901               | 941                  |
| 4 | 24     | 1281              | 1318              | 1368                 |
| 8 | 8      | 578               | 692               | 810                  |
| 8 | 17     | 1655              | 1787              | 1890                 |
| 8 | 24     | 2521              | 2624              | 2743                 |

La columna estándar clásico se corrigió con una búsqueda ampliada, probando los 31 patrones no nulos de fila como semilla en vez de solo los 5 de un único bit. Esto solo mejora, es decir reduce, la cota del estándar clásico: en ese orden  $\pi$  cae entre la semilla y  $\chi$  y dispersa filas multi-bit en sub-filas más baratas; los otros dos órdenes no tienen  $\pi$  ahí, así que no mejoran igual. A partir de 3 rondas el número de pares estimado,  $2^{\text{peso}}$ , ya es inviable para un atacante real. `capa_lineal_rediseño` exige la cota más alta de las tres desde 2 rondas, y la brecha contra el estándar corregido es mucho mayor a 2-3 rondas que desde 8, el mismo patrón que determina el cruce de costo-eficiencia de la Tabla 3.

### 7.3. intentos\_fallidos, rondas, y transparencia de sigma e iota

`rondas = min(24, 8+intentos_fallidos)` satura en 24 cuando `intentos_fallidos ≥ 16`; el rango menor a 10 es el único que nunca alcanza las 24 rondas de diseño original, quedando permanentemente reducido mientras el contador se mantenga bajo. Aun en el peor caso, 8 rondas, la cota ya es de varios cientos, ya que la propia difusión de  $\theta$  y  $\chi$  da margen suficiente incluso ahí. Tanto el término XOR de sigma como el de iota, incluyendo `dominio_aplicacion`, son transparentes ante el diferencial clásico: cualquier afirmación de que dificultarían ataques que necesitan del determinismo no se sostiene. El único efecto real de `intentos_fallidos` sobre esta métrica es el número de rondas y el reordenamiento  $\chi \leftrightarrow \pi$ .

Se probó también una combinación de código no explorada antes, sigma activo, `intentos_fallidos ≥ 3`, junto con `capa_lineal_rediseño` en lugar de  $\theta + \rho + \pi$ , dos caminos que hasta ahora eran mutuamente excluyentes. La cota superior construida coincide con la de rediseño sin sigma en la mayoría de los casos probados, pero en 2 de 12 combinaciones resulta menor:  $z=4$ , 3 rondas, 101 en vez de 103, y  $z=8$ , 8 rondas, 805 en vez de 810. La rotación de sigma sí actúa sobre  $\Delta$ , a diferencia de su término XOR, así que puede alterar el patrón de bits activos que llega a  $\chi$ ; en estos 2 casos lo altera hacia una combinación ligeramente más barata para un atacante. No representa un riesgo práctico, la cota sigue siendo de cientos de bits de peso, pero es una excepción real, no una coincidencia a ignorar.

## 8. CANAL LATERAL

La inyección dinámica no debe introducir debilidades de canal lateral. Se identifican y miden dos vectores en `keccak_f1600_variante`: el tiempo de ejecución, correlacionado con `intentos_fallidos` porque determina el número de rondas, y la bifurcación `if intentos_fallidos < 3` en sí misma, un vector clásico de predicción de rama y consumo energético.

**Tabla 5.**

*Tiempo de ejecución medido según intentos\_fallidos*

| intentos fallidos | rondas | tiempo promedio, 200 corridas |
|-------------------|--------|-------------------------------|
| 0                 | 8      | 204.5 $\mu$ s                 |
| 3                 | 11     | 428.7 $\mu$ s                 |
| 10                | 18     | 687.4 $\mu$ s                 |
| 20                | 24     | 934.4 $\mu$ s                 |

Crecimiento medible y monótono,  $t(20)/t(0) = 4.6\times$ . El salto en `intentos_fallidos=3` supera lo esperado por una sola ronda adicional: ahí también se activa la rama con `sigma`, así que la bifurcación en sí, no solo el conteo de rondas, deja huella en el tiempo.

Si `intentos_fallidos` representa un contador de seguridad real, esta correlación es explotable por un ataque de temporización clásico (Kocher, 1996): un atacante que solo mide tiempos de respuesta puede inferir el contador sin acceso directo, y en particular detectar cuándo el sistema opera con pocas rondas, la región de menor margen ya identificada. El canal lateral agrava esa debilidad: ya no hace falta observar el contador directamente para explotarla. Esta implementación no aplica mitigaciones de tiempo constante, por ejemplo ejecutar siempre 24 rondas o seleccionar la rama sin bifurcación real; se documenta como limitación abierta, no resuelta.

## 9. CONCLUSIONES

Ningún escenario evaluado, ni la variable dinámica ni el rediseño estructural, rompe la propuesta frente a criptoanálisis diferencial clásico, incluso en el peor caso, 8 rondas con `intentos_fallidos` bajo. Las debilidades reales, por relevancia: el número de rondas depende de un contador mutable, reduciendo el margen mientras `intentos_fallidos < 16`; luego, esa correlación es observable por canal lateral de temporización, agravando la debilidad anterior; además, `sigma`, `iota` y `dominio_aplicacion` son transparentes ante diferencial clásico, sin aportar la resistencia que podrían sugerir. También, el perfil IoT/sensor de `dominio_aplicacion` cede margen genérico, 224 de 256 bits, de forma cuantificada. Finalmente, abaratar `chi` por debajo de su costo actual rompe biyectividad o resistencia diferencial; es, dentro de lo

explorado, prácticamente óptima. `capa_lineal_rediseño` cuesta 1.582 veces el hardware por ronda del estándar; rinde más margen de seguridad por compuerta a 2 y 3 rondas y deja de hacerlo a partir de 8, ver Tabla 3, un balance todavía no aprovechado por esta implementación, ya que la fórmula de número de rondas es la misma para ambas variantes.

## REFERENCIAS

- [1] G. Bertoni, J. Daemen, M. Peeters, y G. Van Assche, "Sponge functions," presentado en ECRYPT Hash Workshop, 2007.
- [2] Q. Huangfu y J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018. doi: 10.1007/s12532-017-0130-5.
- [3] P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," en *Advances in Cryptology, CRYPTO '96*, N. Koblitz, Ed. Berlín: Springer, 1996, pp. 104–113. doi: 10.1007/3-540-68697-5\_9.
- [4] S. Mitchell, M. O'Sullivan, y I. Dunning, "PuLP: A linear programming toolkit for Python," documentación de software, 2011. [En línea]. Disponible: <https://coin-or.github.io/pulp/>.
- [5] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," FIPS PUB 202, U.S. Department of Commerce, 2015. doi: 10.6028/NIST.FIPS.202.